Well-Well: Asset Intelligence Copilot

FlytBase AHC SWE Hack Day, Pune · 17 Oct 2026 · Open source (MIT) · Repo: https://github.com/sarthaksingh007/well-well · Run: `docker compose up`, then open http://localhost:8765

Core idea: the AI decides *what to look up and how to say it*, and our code decides *what is true*. Every number in an answer is computed in SQL or Python from the records. Every claim cites the inspection, repair, regulator event, note or manual section it came from.

Page 1: System design

```
Web UI (designed in Google Stitch): chat, user picker, status card / table / line / bar / map / heatmap, agent graph, memory
drawer
   │  POST /api/ask (steps streamed live)        FastAPI, privacy enforced per user in the service layer
Agent harness ── JSON tool protocol ── Gemini 3.8 Flash → 3.5 Flash → 3.5 Flash-Lite → Gemma 4 26B (fallback chain, backoff,
hedged)
   │  guardrails: injection/delete filters · read-only SQL · unknown citations removed · ungrounded numbers flagged
20 typed tools ── status, spills, downtime, aggregate, repairs, RCA, compliance, installed pumps · doc search · SQL · memory
   │
PostgreSQL 16 ── dataset tables · inspection_facts view (corrections applied) · memory · chats · 14,562 BGE-large vectors
Loader ── fleet → manuals/SOPs (rules extracted) → wells, inspections, repairs + reports (facts extracted), regulator events,
notes
```

How the data is stored, indexed and retrieved

- `wellwell load <folder> --dataset test` upserts rows on the dataset's own IDs. The test release merges next to the sample, and loading twice changes nothing.
- **Rules come from the documents, not from code.** From each manual the loader reads the producing threshold (T200 15 °C, T350 20 °C), the alarm limit and the RCA rule. From each free-text repair report it reads the containment hours, the spill class and the compliance table.
- **One source of truth.** The `inspection_facts` view applies the latest correction to each field. It also works out the margin over air temperature, whether the well is producing, and whether a spill still counts after corrections. Every tool and every user reads from it.
- **Document search.** Manuals, SOPs and reports are split by heading. Each section is embedded with BGE-large (`bge-large-en-v1.5`, run locally) and stored in Postgres, keyed by text hash, so nothing is embedded twice. Results are combined with BM25 keyword search (reciprocal rank fusion).
- For questions that no tool covers, there is a read-only SQL fallback, checked against an allowlist.

Tools, and how numbers, documents and notes are combined The tools are typed and take parameters (any well, area, well type, period or metric), so they don't depend on the exact wording of a question. Sources are combined inside the tools:

- **Downtime** = 7 days for each weekly inspection below the manual's threshold. Each stop is explained by the repair it triggered, or by a note covering that date.
- **SR-02** compares the discovery time with containment hours that appear only in the report text.
- **SR-01, SR-03 and RC-04** compare regulator events with the SOP deadlines.

Memory, in two levels

- **Shared:** notes, corrections and claims, stored per well or per well type with the author and the reason. Every user's later answers use them.
- **Learning from feedback:** a correction to raw data or to the AI's analysis keeps the original value, plus who changed it, why and when. A newer correction replaces the older one in answers, but both stay on record. Team rules (for example "T350: RCA after 2 repairs") apply to that well type only.
- **Private:** each user's conversations, the wells mentioned in them, and their "just for me" display preferences (e.g. sort by area, costs in $k). Recall only searches the signed-in user's chats.
- **Self-evolving playbook:** each verified answer saves an anonymised question pattern and the tool plan that answered it. A 👍 or a passed eval raises its score, and a 👎 marks it to avoid. Similar new questions get the saved plan as a hint. It has an on/off switch.

Conflicts and new data

- A second-hand claim ("Nathan told me…") is checked against what that person actually told the copilot; it is never acted on as fact.
- A user's claim is checked against the latest inspection. If the data contradicts it, the claim is kept as *unconfirmed*, both sides are shown, and the user is asked to confirm.
- Opinion notes never override data; only formal corrections do. An out-of-date recorded status is flagged.
- New data goes through the same loader (`wellwell load-github` pulls the test release), the rules are re-read, and only new text is embedded. The test release's new fields (T500, plugged wells, structured data corrections, recall notices, MOC memos, operator notes) loaded without schema rewrites; recalled pumps come from the "Parts installed" lines in report text. Anything the parser can't read is listed as a load warning, not guessed.

Models and cost

- **Answers:** Gemini 3.8 Flash, then 3.5 Flash, 3.5 Flash-Lite and Gemma 4 26B, all on the free tier of the Gemini API. A model whose daily quota runs out is skipped for the rest of the run.
- **Embeddings:** BGE-large, running locally.
- **Speed:** 1 to 3 model calls per question; the median answer takes about 4 s.
- **Cost:** USD 0.

What we would do next: row-level security per team, proactive alerts (new stops, SR deadlines), checking AI spill estimates against the inspection images, and a review queue for claims and corrections.

Page 2: Results

Sample dataset: the 18 example scenarios

18/18 scenarios and 27/27 steps correct, run end to end through the API on a fresh database. "Correct" means the facts, IDs, dates, names, cited records and the right visual. The median answer took 4 s. We also have 95 automated tests (unit, edge cases, API, loader, embeddings, guardrails), and the Docker from-scratch smoke test passes.

#	Question (short)	Key result	Visual
Q1	WELL-0001 producing now?	No: INS-0001-0203, 15 Oct, Ryan Brooks; 3.2 °C < 15; CTX-0001-003 planned shut-in; recorded status out of date	card
Q2	NE Block 8 not producing, and why	0164, 0170 (MNT-0170-031), 0172 (MNT-0172-028), 0175; two with no recorded reason	table
Q3	13-week temperatures	13 readings; not producing on 8 and 15 Oct	line
Q4	WELL-0001 spills, 4 years	6 spills, 15.1 bbl; INS-0001-0070 excluded (rainwater)	table
Q5	Days down in 2025	21 days (3 × 7): MNT-0001-021, MNT-0001-023	text
Q6	NE-7 top 3 by downtime, 4 months	0121 49, 0138 42, 0107 42 days	bar
Q7	NE-8 real spills, 90 days	0151, 0165, 0175, 0186 with coordinates	map
Q8	2026 spills, area × month	35 / 43 / 37 / 26	heatmap
Q9	WELL-0001 rules broken	SR-01 (41 h), SR-02 (7 h), SR-03, RC-04; USD 5,000 open; RCA and shut-in pending	table
Q10	SW-24 containment over limit	0082 7.1 h, 0100 8.1 h (from reports)	table
Q11	"The three wells from yesterday": cost	USD 25,794 / 46,942 / 5,176 (private recall)	table
Q12	Correct the WELL-0164 reading	24.0 → 41.0 °C, now producing; "Pump jack not moving" note flagged	card
Q13	Spill 4.5 → 5.4 bbl	Now reportable; SR-01 overdue	card
Q14	T350 RCA rule	Adds WELL-0120 (T350), not 0102 (T200); cites Michael Turner	table
Q15	Correction reaches other users	Jessica sees 41.0 °C by Michael Turner, plus the disagreeing note	card
Q16	Shared vs private	Michael sees Jake's WELL-0007 note; Jessica: "no such earlier conversation"	card
Q17	People vs data	Tyler's "running" contradicted (0.4 °C); Jake's WELL-0001 claim unconfirmed	card
Q18	WELL-0101 spill real?	Yes: opinion notes disagree, no formal correction, so it stands	card

Final questions on the test dataset

Test data was loaded with `wellwell load <path> --dataset test`.

#	Question	Our answer (short)	Visual	Notes
F1				
F2				
F3				

What worked

- Deterministic tools made answers exact and repeatable. The model never does arithmetic.
- Reading thresholds and deadlines from the manuals and SOPs, instead of hard-coding them.
- A single corrected view meant every correction reached every answer and every user.
- Prefetching obvious lookups and switching to a faster model cut typical answers from 40–130 s down to 2–10 s.

What didn't, and why

- The free Gemma endpoint often returned 500/504 errors. We fixed this with backoff, hedged requests and a model fallback chain.
- A Postgres NULL-ordering difference picked the wrong downtime cause. Our eval caught it, and explicit ordering fixed it.
- Private recall first fell back to an unrelated chat. It now needs a real match, and otherwise says there is no such conversation.